Defending Yourself from Code Injection

How can you protect your code from code injection? You might parse all *Strings of Code (141)* to identify operations that are potentially dangerous. This approach may prove ineffective, though, because there are so many possible ways to write malicious code. Trying to outsmart a determined hacker can be dangerous to both your computer and your ego.

When it comes to code injection, some strings are safer than others. Only strings that derive from an external source can contain malicious code, so you might simply limit your use of eval to those strings that you wrote yourself. This is the case in *The REST Client Example*, on page 141. In more complicated cases, however, it can be surprisingly difficult to track which strings come from where.

With all these challenges, some programmers advocate banning eval altogether. Programmers tend to be paranoid about anything that might possibly go wrong, so this eval ban turns out to be a pretty popular choice. (Actually, we're not paranoid. It's the government putting something in the tap water that makes us *feel* that way.)

If you do away with eval, you'll have to look for alternative techniques on a case-by-case basis. For an example, look back at the eval in *The REST Client Example*, on page 141. You could replace it with a *Dynamic Method (51)* and *Dynamic Dispatch (49)*:

ctwc/rest_client_without_eval.rb

```
POSSIBLE_VERBS.each do |m|
  define_method m do |path, *args, &b|
    r[path].send(m, *args, &b)
  end
end
```

In the same way, you could rewrite the Array Explorer utility from *Code Injection*, on page 146, by using a Dynamic Dispatch in place of eval:

ctwc/array_explorer_without_eval.rb

```
def explore_array(method, *arguments)
  ['a', 'b', 'c'].send(method, *arguments)
end
```

Still, there are times when you might just miss eval. For example, this latest, safer version of the Array Explorer wouldn't allow your web user to call a method that takes a block. If you want to describe a Ruby block in a web interface, you need to allow the user to insert arbitrary Strings of Code.